A Rescue That Knows Its Group: Training Deadlocks and Partial-Credit Traps in Learning to Compose from Pixels

Sankalp Pathak[1], **Sanjay Garg**[1], **Naveen Saini**[2]

[1] Jaypee University of Engineering and Technology, Guna, Madhya Pradesh, India

[2] Indian Institute of Information Technology Allahabad, Prayagraj, Uttar Pradesh, India

Abstract

Small vision transformers trained to combine symmetries, such as the rotations and reflections of a square, shown as pictures, often never learn the task: most failing runs give the same answer to every input for the whole of training. We study why, on a benchmark where the same group of symmetries can act either on where a glyph sits or on what colours it has. We find three things. First, a failing run is stuck, not incapable. A probe that reads the answer inside a model that solved the task reads nothing at the same places inside a failing one. And a failing task is learned in every seed when training starts from a small model that already knows a one-step version of the same task, in nearly every seed when that model learned the same group on the other kind of picture, and no more often than from scratch when it learned a different group. That pattern held twice on fresh seeds whose tests were written down before the runs. Second, a visible cue that gives away one true bit of the answer traps training: runs that pick it up stop at the score that bit alone earns, a prediction that also held on fresh seeds. Third, how far apart a rendering draws the group's elements, measured from the renderer and fixed before it was compared with anything, tracks how often seven renderings are learned. The group was learned far more often from position than from colour at the setting we started with, but that gap was not shown to survive a change of learning rate and weight decay or of glyph size, so we report it as specific to that setting. Looped and feedforward models fail at rates we could not tell apart, so none of this is about recurrence.

1. Introduction

To combine two symmetries shown as pictures, a network has to recognise each picture, keep track of which operation it stands for, and apply the two in the right order (**Figure 1**). The result is a single symmetry, which we call the composite. We built a benchmark where the number of operations is known exactly and the same group can act on two different things: where a glyph sits, or what colours it has (**Figure 2**a). Everything else about the group stays the same. We set out to test whether running one block of a vision transformer several times, which is what a looped model does (**Figure 2**b), helps with this kind

of task. At matched compute it does not. What we found instead is about how such models fail to learn, and what gets them out.

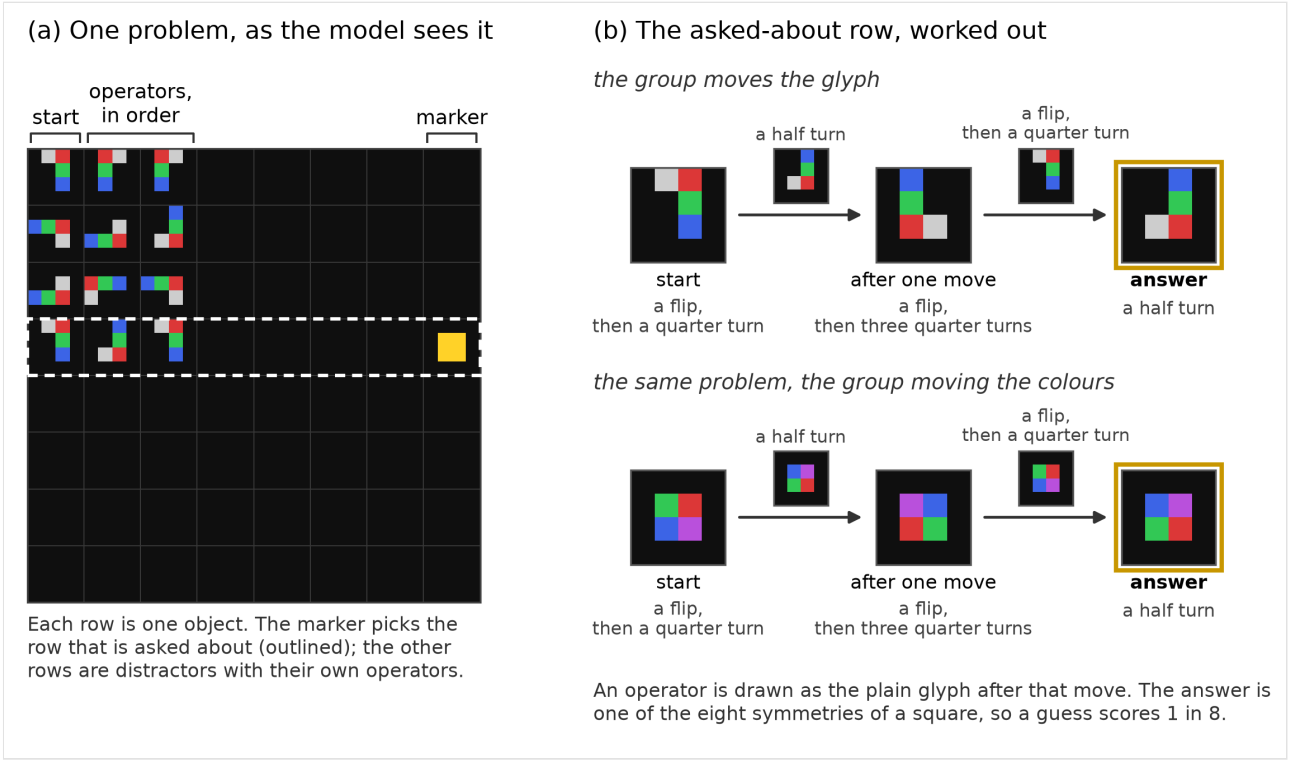


Figure 1. One problem, read the way the model has to read it, drawn by the generator and the renderer. (a) A depth 2, breadth 4 problem on its 8 by 8 grid of 4 by 4 pixel patches. Each row is one object: its starting state, then its two operators in the order they apply, and the yellow marker on the row that is asked about (outlined). The other rows are distractors. (b) That row worked out with the group's own arithmetic: each operator applied in turn, ending at the answer, one of the eight symmetries of a square. Below, the same problem drawn with the group acting on colour: the same moves and the same answer, and every tile has the same outline. The sample is the first of the validation split whose asked-about row has no step that does nothing, a rule fixed in the plot script.

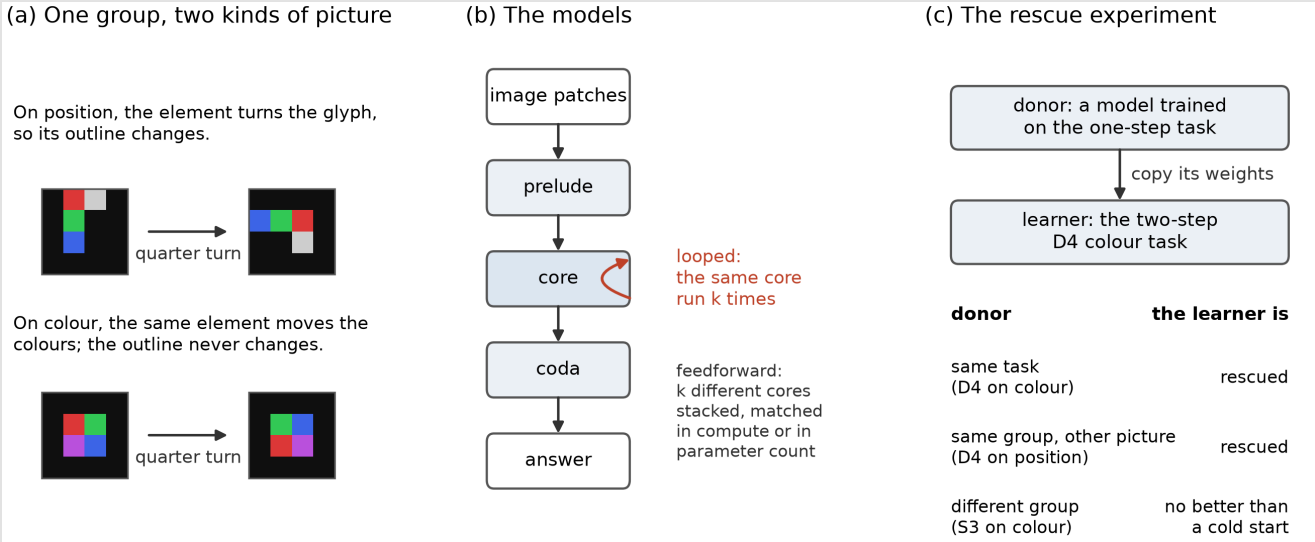


Figure 2. The setup as a schematic. (a) The renderer's own tiles for doing nothing and for a quarter turn: on position the glyph turns and its outline changes; on colour the colours move and the outline stays the same. (b) The looped model runs one core k times between a prelude and a coda; the feedforward baselines stack k different cores instead. (c) The rescue experiment of Section 5.3: a donor trained on the one-step task gives its weights to a learner on the two-step D4 colour task.

Most runs that fail do so the same way. They sit at chance, giving one answer to every input, for the whole of training (**Figure 3**, top left). This paper is about that failure and makes three contributions.

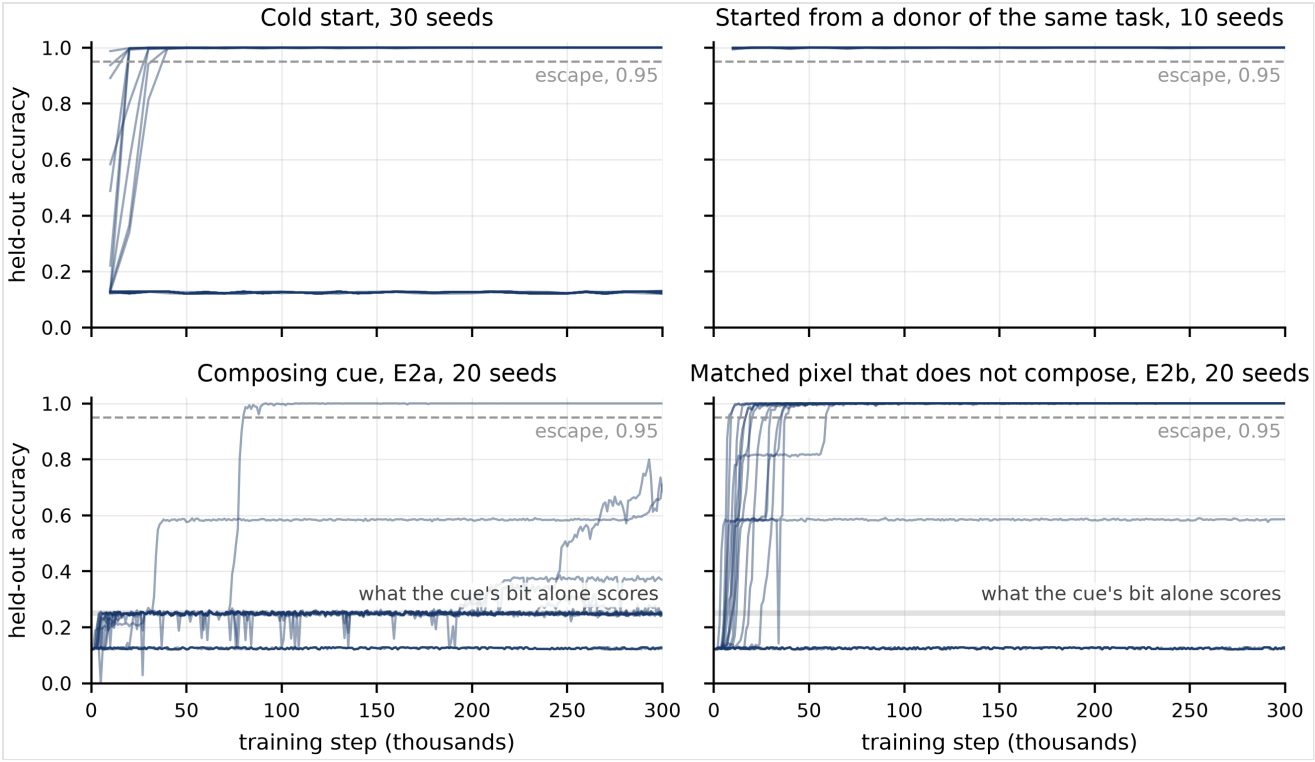


Figure 3. Held-out accuracy against training step for every seed of four arms, one line per seed, with nothing selected. Top left, the D4 colour task from a cold start: each run either rises early or stays on its floor to the end. Top right, the same task started from a donor that learned its one-step version: every seed is already solved at the first evaluation, at step 10000. Bottom, the D4 colour task with a cue that carries one composing bit of the answer (E2a, left) and with a matched pixel that does not compose (E2b, right). The grey band, 0.24 to 0.26, is the band later registered for the fresh seeds of Section 5.4; these are the first-set runs it was drawn from. The top row was evaluated every 10000 steps and the bottom row every 1000.

1. **A deadlock, and a rescue that knows its group.** A failing run does not compute the composite at all: a probe that reads it inside a solved model reads nothing at the same places inside a failing one. Starting training from a donor, a small model trained first on a one-step version of the same group, breaks the deadlock within a few thousand steps (**Figure 3**, top right). What the rescue needs is the right group, not the right picture: a donor that learned the same group on the other kind of picture works, and a donor that learned a different group does not (**Figure 2**c). Both halves held on two sets of fresh seeds whose tests were fixed before the runs existed (**Figure 4** draws every seed). Section 5.3.

2. **A true partial answer that traps training.** We added to the colour pictures a visible cue that gives away one bit of the answer, and that bit is always right. It did not help. Runs that pick it up stop at the score that bit alone earns (**Figure 3**, bottom left), and that held as a registered prediction on fresh seeds. A shortcut, in the sense of shortcut learning, is a rule that works on the benchmark but fails to transfer (Geirhos et al., 2020). This cue is correct everywhere, so the trap is partial credit on the real task (**Figure 5** shows why). Section 5.4.

3. **How far apart the pictures are tracks how often they are learned.** We defined, built and tested a measure of how far apart a rendering draws the group's elements, in pixels, before computing it on any rendering. Its rank correlation with how often the task is learned, across seven

renderings, cleared the threshold fixed in advance (**Figure 6**). With seven points this is suggestive, not significant. Section 5.5.

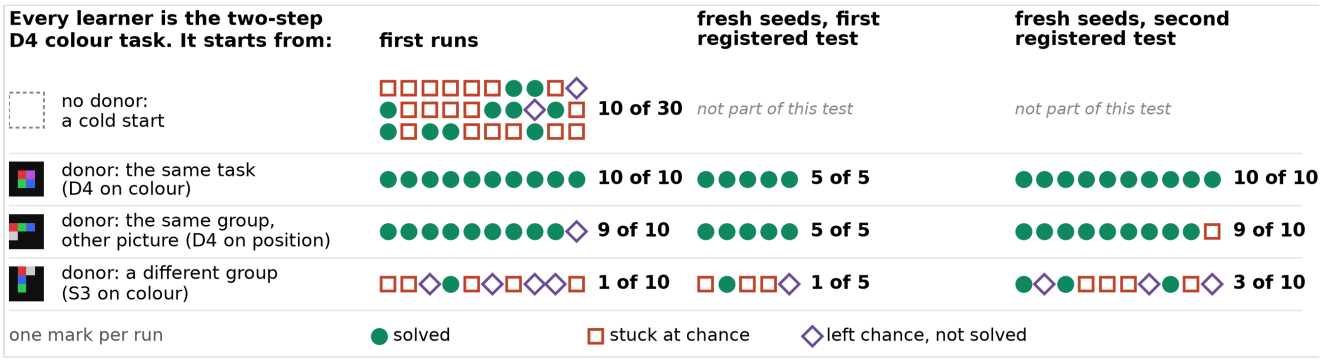


Figure 4. Every seed of the rescue experiment, one mark per run, by its stored outcome: solved, stuck at chance, or left chance without solving. Every learner is the two-step D4 colour task; the rows are what it started from, with the donor's own tile at the left, and the columns are the first runs and the two fresh sets whose tests were registered before the runs. Both donors of the right group rescue nearly every seed in every set; the donor of a different group rescues few, and in the first runs no more often than a cold start, which only the first runs include. Outcomes and counts are the stored ones (Appendix A).

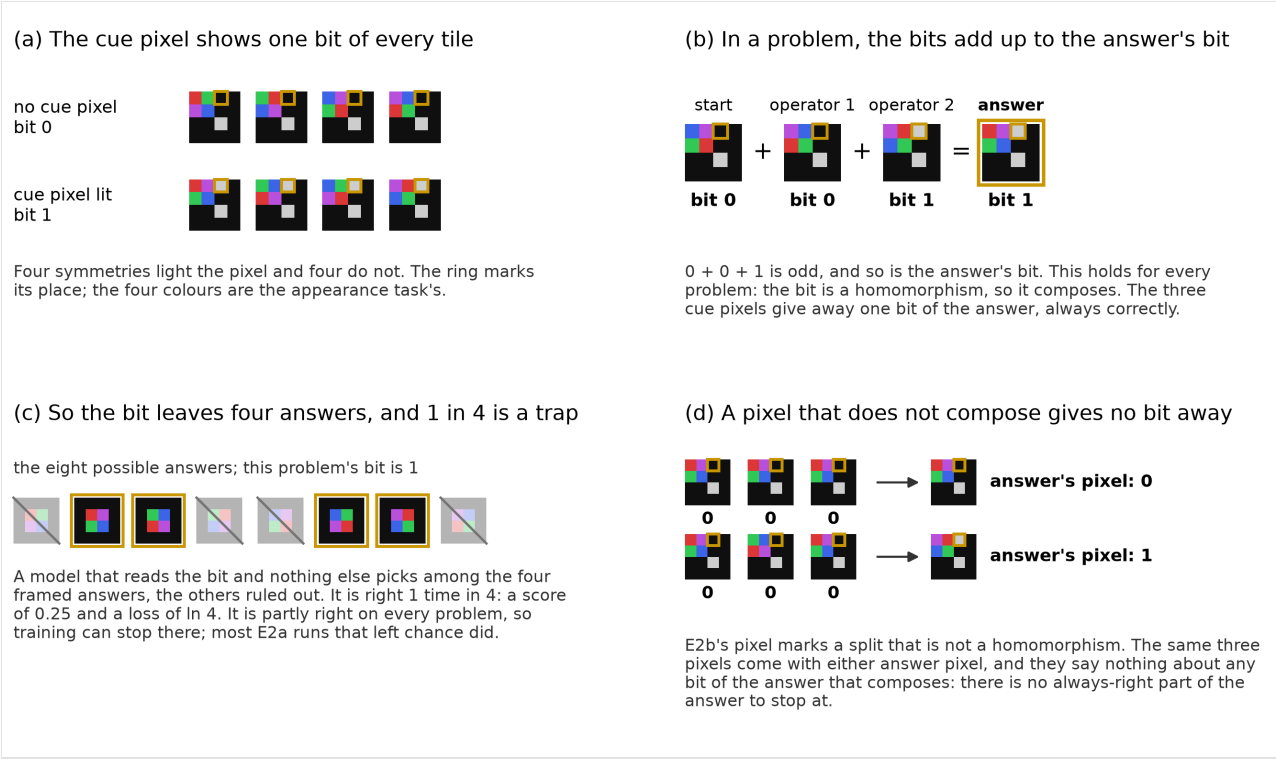


Figure 5. How a true bit becomes a trap, worked out on the renderer's tiles with the group's own arithmetic; no measured number is drawn. (a) E2a's cue pixel, ringed, is lit on four of the eight D4 tiles. (b) In a real problem the three cue bits add up, counting in twos, to the bit of the answer; they do in every problem, because the bit composes. (c) So a model that reads the bit and nothing else is left with four of the eight answers and is right one time in four, a score of 0.25 and a loss of ln 4. (d) E2b's pixel marks a split that does not compose: the same three pixels come with either answer pixel, and they carry nothing about any bit of the answer that composes. Each example is the first, in a fixed order, that meets a rule stated in the plot script.

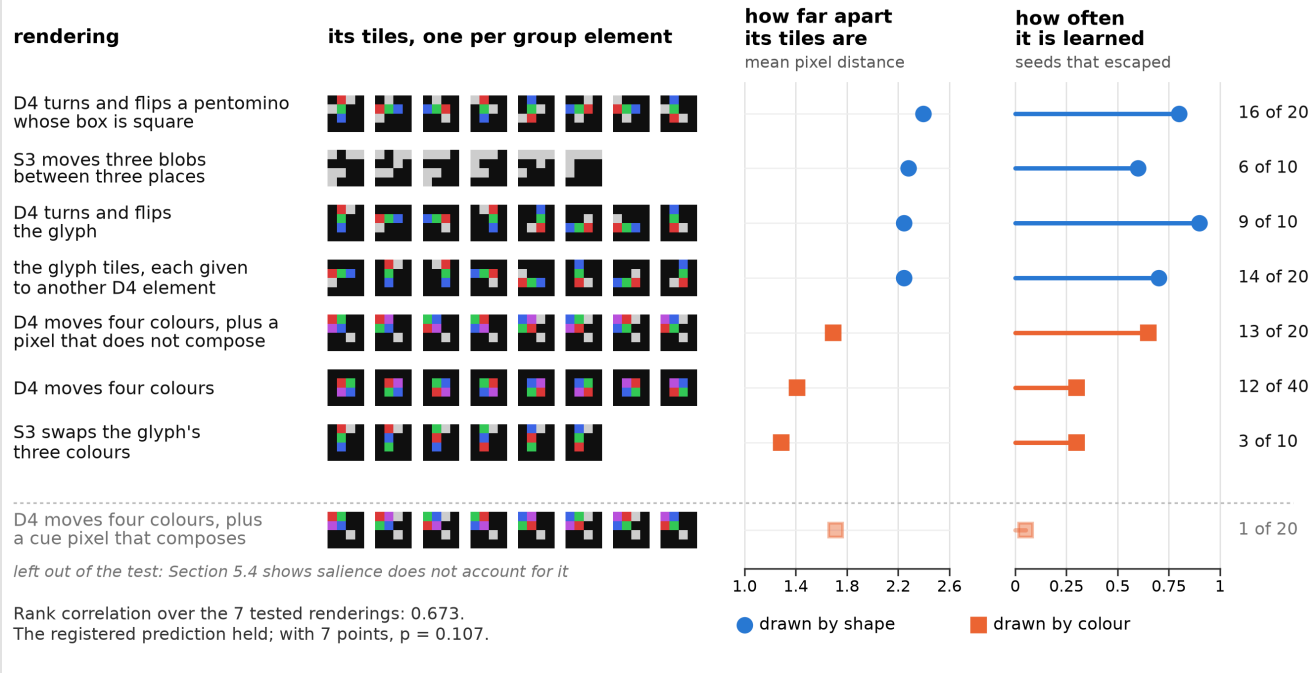


Figure 6. Eight renderings of a group, one per row, with every tile drawn by the renderer: how far apart its tiles are, as the mean pixel distance between them, and how often the task drawn that way is learned. Rows are sorted by distance. Circles mark renderings drawn by shape and squares those drawn by colour. The composing cue of Section 5.4, below the rule, was left out of the registered test. Distances, counts and the rank correlation are the stored ones (Appendix A).

The difference that started the study, that the group is learned far more often from position than from colour, is real at the setting we first used and held there on fresh seeds. It was not shown to survive a change of learning rate and weight decay, or glyphs spread over several patches, and Section 5.2 reports it with those limits. Every test in this paper was registered, that is, written down and committed before its runs existed, unless it is labelled post hoc.

2. Related work

Universal transformers (Dehghani et al., 2018) are recurrent in depth, and recurrent-depth language models (Geiping et al., 2025) iterate a shared block to reach arbitrary depth at test time. Recurrent networks that iterate longer at test time extrapolate to harder problems than they were trained on (Schwarzschild et al., 2021; Bansal et al., 2022). None of these tests whether recurrence helps binding specifically, and we found no prior work that does. Geiping et al. (2025) stop iterating when the KL divergence between successive steps is small. Pappone et al. (2025) propose an exit rule on the second-order difference of the step size, which they report beats that KL exit and a first-order rule on the step size itself. Popescu et al. (2026) find that simple confidence readouts often match or outperform learned halting gates. Our benchmark knows how many passes each item needs, and Appendix B scores such rules against that.

Greff et al. (2020) set out the binding problem for artificial networks. Huang et al. (2026) formalise it information-theoretically and measure binding information in pretrained vision transformers; their

abstract does not describe manipulating spatial and appearance features independently on a fixed group, which is what our benchmark does.

Shortcut learning (Geirhos et al., 2020) names decision rules that perform well on standard benchmarks but fail to transfer, and simplicity bias (Shah et al., 2020) names networks that rely on the simplest predictive feature and ignore more complex ones. Hermann and Lampinen (2020) show that an easier, weakly predictive feature can suppress a more strongly predictive one, and that a model's preference tracks what is most linearly decodable from the untrained model; Hermann et al. (2023) separate a feature's predictivity from its availability. The trap of Section 5.4 is a case of an available feature winning, with one difference: the feature is not spurious but a true part of the answer, so the model that takes it is right about part of the answer and then stops. Our salience measure in Section 5.5 is close in spirit to an availability measure, but it is computed from the renderer rather than from a network.

Networks trained on finite group operations learn circuits built from the group's representations, and which circuits a given network learns, and in what order, is arbitrary (Chughtai et al., 2023). Low-depth transformers can represent the computations of finite-state automata, of which composing group elements is one case, through shortcuts (Liu et al., 2022). Delayed generalisation on small algorithmic tasks is the grokking literature (Power et al., 2022; Nanda et al., 2023); our benchmark's validity gate saw it directly, in a first training budget that stopped before the task was learned.

Control tasks (Hewitt and Liang, 2019) measure a probe's selectivity by giving it random targets. Our control asks a different question: whether the same probe, at the same sites, finds the information in a model known to contain it. Only that second control lets a null probe on a failing model count as evidence that the information is absent. The coda lens we use is Lu et al.'s (2025), and our appendix result with it agrees with their finding of limited evidence for an interpretable latent chain of thought, here on images rather than text.

---

3. The benchmark

Each problem is drawn entirely in pixels: a scene, and a short sequence of group elements to apply to it. Three families of problems share one vocabulary and one renderer.

- **Family A, depth:** a composition of group elements applied to a scene. Depth is the number of operators composed. The answer is the composite, with the order respected.
- **Family B, depth with breadth held fixed:** used to measure how many passes of the loop each depth needs (Appendix B).
- **Family C, breadth:** 4 to 8 objects at depth 1, so the amount of parallel work varies while the number of sequential steps does not.

The same group can act on either of two things, which we call substrates. In the *position* condition the group moves where a glyph sits and how it is turned. In the *appearance* condition, written `identity` in

the code, the same group moves the glyph's colours around. The group's order, its generators, its abelianisation (its largest commutative quotient) and the order-blind ceiling defined below are the same in both. The only thing that differs is what the group acts on, and at the level of pixels that is a lookup table of eight (D4) or six (S3) small rendered tiles: in the position condition each element has its own outline, and in the appearance condition every element has the same outline and differs only in where its colours sit (**Figure 2**a). Every glyph fills one 4 by 4 patch of a 32 pixel canvas.

We use D4, the eight symmetries of a square, and S3, the six permutations of three things. D4 is generated by a quarter turn and a reflection about the main diagonal. Some pairs of elements give the same composite in either order, so a model can score above chance without tracking order. The order-blind ceiling is the best accuracy a model can reach if it knows which operators appeared but not in what order. At depth 2 it is 0.8125 for D4 and 0.7500 for S3, computed exactly over all operator pairs before any run. A score above the ceiling is evidence that the model composes in order, not just that it knows which operators were present.

Figure 7 shows the benchmark itself, in real samples from the generator. Family A raises the operator count from 1 to 3 at a fixed breadth of 4, family C raises the object count from 4 to 8 at a fixed depth of 1, and the third row puts the two substrates side by side at matched depth and breadth. **Figure 1** works one family A problem through by hand, and **Figure 6** shows the eight renderings that Sections 5.4 and 5.5 compare, tile by tile.

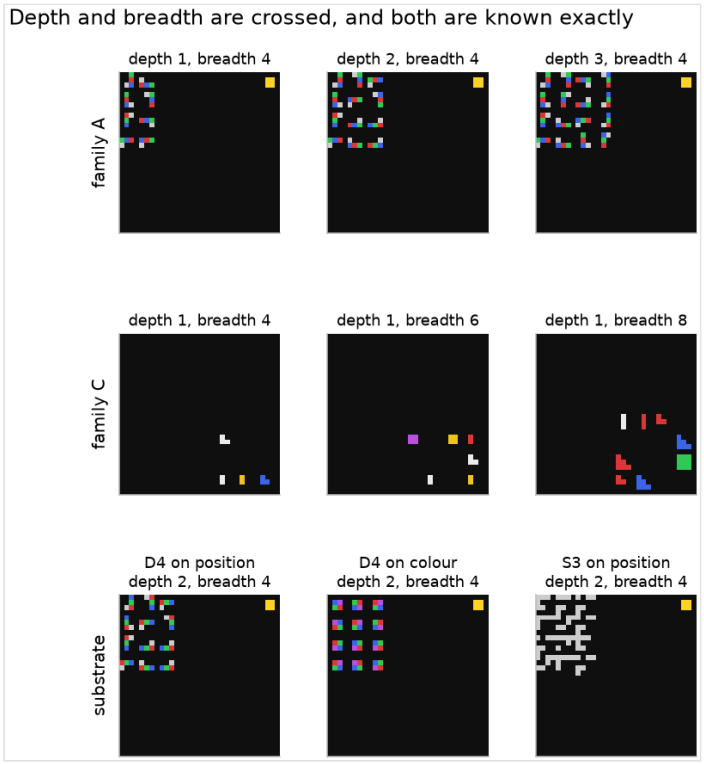


Figure 7. Real samples from the generator at a fixed seed, each labelled with the depth and breadth read off its own program. Top, family A at depths 1 to 3 and breadth 4. Middle, family C at breadths 4 to 8 and depth 1. Bottom, the same group acting on position and on colour at matched depth and breadth; this row needs colour to read.

4. Models and training

Every model is a vision transformer built from three parts: a prelude that reads the image patches, a core, and a coda that reads out the answer (**Figure 2**b). The looped model runs the same core k times. Two feedforward baselines instead stack k different core blocks, one matched to the looped model's compute and one to its parameter count. All three share the prelude, coda, output norm and head, so they differ in weight sharing and depth, not in how they take in the image or give the answer.

Unless stated, models have width 384 and 2, 2 and 2 blocks in prelude, core and coda, about 11M parameters for the looped model, and train for 300000 steps at learning rate 3e-4 and weight decay 0.05. Runs added for the robustness and confirmation tests of Sections 5.2 to 5.4 stop early once validation accuracy has stayed at or above 0.95 for 20000 steps. That rule was fixed before those runs and checked on earlier runs, none of which finished below 0.95 after reaching it.

We say a run escapes if it finishes at or above 0.95 held-out accuracy. Nearly every run finishes either there or on a floor near chance, so the rates in this paper count escapes, and no failed run is left out of them.

A perfect score gets three controls: a blank-image control, an input ablation, and the analytic ceiling above. The solved D4 model needs its state and both operators, correctly ignores distractors, and its 1.0000 sits above a computed ceiling of 0.8125.

The benchmark was validated before anything was measured on it, and passed on its second attempt. A depth 3 task was unsolvable at one pass (0.0212 against a 0.0208 chance floor), a depth 1 task was solvable at one pass, the order-blind ceiling sat below the threshold at every depth, and family C was solvable at every breadth. The first attempt failed only because its budget stopped short: the task was learned abruptly at about step 84000, the delayed generalisation known as grokking (Power et al., 2022).

---

5. Results

5.1 What a failing run looks like

Of 196 finished family A runs at the time we counted, 104 finished above 0.95, 91 sat on a floor, and one landed between. Of the finished depth 2 failures that recorded it, 72 of 78 emitted exactly one class for all 16384 evaluation items, so the floor accuracy is the frequency of that constant class and not a measurement of anything the model computes. Escape is close to all or nothing, which is why we report rates of it. **Figure 3** shows every seed of four arms. In the cold-start arm at its top left, each run either rises above 0.95 early or stays on its floor to the end.

5.2 The motivating asymmetry, and its limits

At the setting described in Section 4, the same group is learned far more often when it acts on position than when it acts on colour. Three competing predictions were registered before either new cell was built.

The 2x2 is complete: 70 of 70 seeds finished. Every cell carries at least the ten seeds the preregistration requires of a number that reaches a headline.

	acts on position	acts on identity
D4, order 8	9 of 10	12 of 40, floor 0.1210 to 0.1294
S3, order 6	6 of 10, floor 0.1681	3 of 10, floor 0.1681 to 0.1787

Identity is 15 of 50 finished across both groups and position is 15 of 20.

A two-sided Fisher test gives p = 0.0011. At this setting the escape rate follows the substrate and not the group: the registered alternative, in which the group's abelian part sets the rate, predicted the opposite in both new cells and is refuted, as is the interaction alternative. **Figure 8** plots one point per seed rather than a mean, because the cells are not uniform. Fresh seeds held out of this grid, five for each of three cells, do not contradict it.

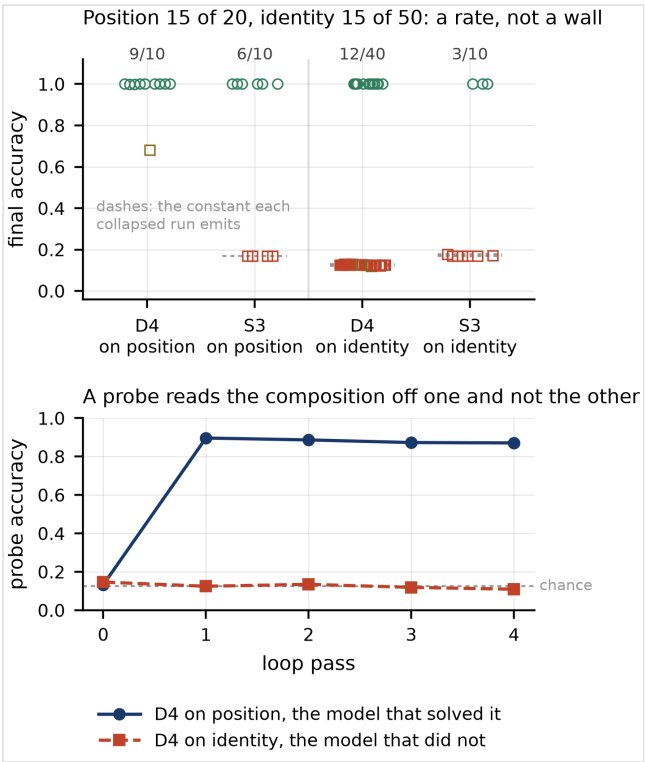


Figure 8. Top, final accuracy by cell of the 2x2, one point per seed. Bottom, a probe's readout of the composite across passes, in a position run that solved the task and an appearance run that did not. Rates are at learning rate 3e-4, weight decay 0.05 and one glyph per patch; the figure writes the appearance condition as identity, its name in the code.

The asymmetry is not about loops. At matched compute the looped model reached 0.9966 and the feedforward baseline 0.9998, and the gate that required looping to win by 0.02 failed. On the D4 appearance task the looped, matched-compute and matched-parameter models escape in 10, 7 and 10 of 30 seeds, with every pairwise Fisher p at or above 0.567.

Two tests, registered before their runs, asked whether the asymmetry survives a change that should not matter. It was not shown to survive either.

change	prediction	outcome
six learning-rate and weight-decay settings, five seeds a side each	position escapes more often, pooled over settings	**failed**: 10 of 30 against 7 of 30, stratified p = 0.4008
the same, any single setting	appearance never escapes significantly more often	held
glyphs drawn at twice the size, so each spans 2 by 2 patches	position escapes more often	**failed**: 10 of 10 against 7 of 10, p = 0.211

At weight decay 0.5 one run in thirty escaped on either substrate, so half the grid carried little information. That cost the test power, and its registered verdict stands. At the original setting the grid's own five seeds a side went 4 of 5 against 2 of 5. As registered, the asymmetry is shown only at learning rate 3e-4, weight decay 0.05 and one glyph per patch; at no setting did appearance escape significantly more often.

5.3 A training deadlock with a rescue that knows its group

In a failing run the composite is absent, not undecodable. A trained probe reads the composite at 0.896 off the states of a model that solved the D4 position task, and reads chance off a model that failed the D4 colour task, at the same sites and with the same code. A null probe is evidence that the information is absent only if the same probe finds it when it is present. **Figure 9** puts the lens row, the probe row and that positive control on one panel for that reason.

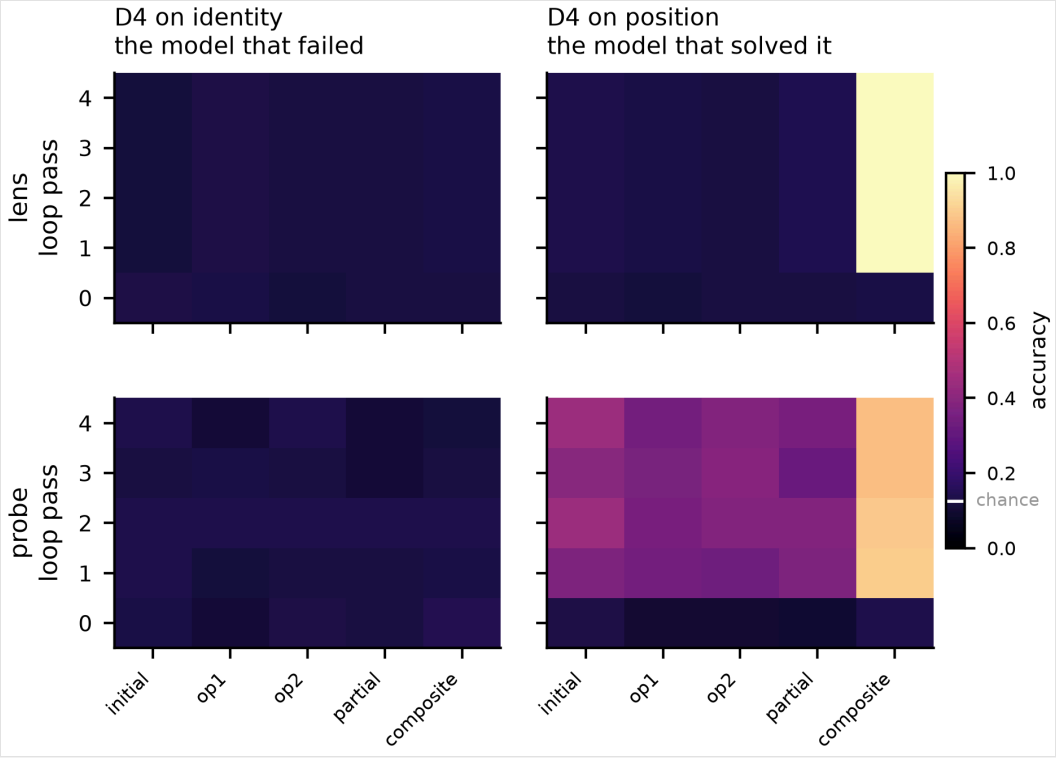


Figure 9. Absent, not undecodable. The same probe at the same sites reads the composite in the solved model and nothing in the failing one; the coda lens row shows the same split.

A donor breaks the deadlock. A donor is a model trained first on the depth 1 version of a task; the learner starts from a copy of its weights instead of from random ones (**Figure 2**c). A depth 2 appearance task that a cold start escapes in 10 of 30 seeds is solved in every one of ten seeds started from a depth 1 donor of itself, each seed from its own donor, and every one of them left chance within about 2000 to 3000 steps (**Figure 3**, top right).

donor, a depth 1 model of	group	substrate	escaped	Fisher p against the wrong-group donor
the same task	D4	appearance	10 of 10	below 0.0005
the same group on the other substrate	D4	position	9 of 10	0.001
another group	S3	appearance	1 of 10	
no donor, cold start			10 of 30	0.233

The control that gives this meaning is the wrong-group donor, matched on architecture, prior training and task family and differing only in group. It rescues no more often than a cold start, so the rescue is not generic warm starting. What transfers is the group, not the substrate: the two donors that rescue disagree about substrate, and the one that does not agrees with the task about substrate and differs in group. So, at the setting where the asymmetry holds, how often a task is learned from cold depends on the substrate, and whether a foothold transfers depends on the group.

Both halves held twice on fresh seeds registered in advance. **Figure 4** draws every seed of the table above and of the two tests below, one mark per run, so the pattern can be read without the counts.

test	same group, same substrate	same group, other substrate	different group	verdict
first set, five seeds an arm	5 of 5	5 of 5	1 of 5	both held, each p = 0.048
second set, ten seeds an arm	10 of 10	9 of 10	3 of 10	both held, Holm-adjusted p = 0.006 and 0.020

Each test compares one rescuing arm with the wrong-group arm on seeds that did not exist when the test was written. The first set held at the narrowest margin that could pass at five seeds a side; the second, with twice the seeds and a Holm correction across the two tests, held with room to spare. In the second set three wrong-group recipients escaped, two of them within about 2000 steps, a donor's timing rather than a cold start's, and one other-substrate recipient never left chance; both are counted.

If composing appearance were out of the model's representational reach, a donor could not fix it, and that argument does not depend on the rates above.

5.4 A true partial answer that traps training

Our first explanation for why position is easier was a staircase, and it failed. Only D4 acting on position climbs a staircase: in 9 of its 10 sealed seeds the training loss sits near ln 4, the level of knowing the answer up to four candidates, for at least 10000 steps before the solve. The reason is visible in the pictures. D4 can be mapped onto a single bit in three ways that respect composition (its three homomorphisms onto the two-element group), and in the position rendering one of those bits, rotation plus reflection mod 2, can be seen in the tile's shape: with the reflection about the main diagonal, the glyph stands tall exactly when that sum is even. We registered the hypothesis that a visible bit like this gives the model partial credit to climb from. Appendix C reports its tests; the one registered as decisive gave the appearance rendering exactly such a bit.

E2a adds to the appearance rendering a fixed anchor pixel and a cue pixel that is present exactly when that bit is odd. The cue therefore composes: the three cue bits of a problem together carry one bit of its answer. E2b is identical except that its pixel marks something that does not compose. **Figure 5** works this through on the real tiles. The cue bits of every problem add up to the bit of its answer, so a model that reads them and nothing else is left with four of the eight answers and is right one time in four. E2b's pixel carries nothing about any bit of the answer that composes.

	composing cue, E2a	**matched non-composing pixel, E2b**
first set, twenty seeds each: escaped	1 of 20	13 of 20
first set: runs that left chance and climbed to ln 4	14 of 15	0 of 14
second set, ten seeds each: escaped	3 of 10	4 of 10

The trap held as a registered prediction. In the first set, eleven of the fifteen E2a runs that left chance finished between 0.2455 and 0.2495, which is what a model scores if it reads the cue's one bit and nothing else, and one solved (**Figure 3**, bottom left; the matched E2b pixel is at bottom right). We then registered that at least half of a fresh set's E2a runs that left chance would finish between 0.24 and 0.26, and five of eight did.

In the first set the composing cue escaped far less often than the non-composing pixel. A registered test that this reversal would recur on fresh seeds failed: E2a 3 of 10 against E2b 4 of 10. So the claim here is about where runs that take the cue stop, not that the cue lowers the rate of escape.

This is not ordinary shortcut learning. A shortcut in the usual sense is a feature correlated with the label in the training data and wrong outside it. This cue is a correct quotient of the label everywhere, so a model that uses it is right about part of the answer. The setting has an exact algebraic description of which partial answers exist, and the runs stop at the one that is visible. Our own registered hypothesis expected a visible quotient like this one to help (Appendix C); it gave training a place to stop instead.

5.5 Salience, measured before it was correlated, tracks learnability

How far apart a rendering draws the group's elements was defined before it was computed: the mean, over all pairs of distinct elements, of the Euclidean distance between their rendered tiles in pixels scaled to 0 to 1. The same distance after a freshly initialised model's own patch embedding, averaged over 20 initialisations, was registered as a secondary. The prediction was a Spearman correlation with escape rate of at least 0.6 across seven renderings, with the composing cue left out because Section 5.4 already shows salience does not account for it. **Figure 6** draws each rendering's tiles beside its distance and how often it is learned.

rendering	distance between tiles, pixels	after the patch embedding	escaped
D4 position	2.2455	0.8804	9 of 10
D4 appearance	1.4104	0.5515	12 of 40
D4 moving a pentomino whose box is square	2.3948	0.9391	16 of 20
D4 position tiles assigned to other elements	2.2455	0.8804	14 of 20
D4 appearance with a non-composing pixel	1.6937	0.6602	13 of 20
S3 position	2.2787	0.8875	6 of 10
S3 appearance	1.2868	0.5005	3 of 10

The correlation is 0.673, and the prediction held. D4 position and the rendering that reassigns its tiles draw the same eight tiles, so their distances are equal and the two share a rank, as registered. After the patch embedding the correlation is also 0.673, reported and not tested. Renderings whose elements look further apart are learned more often. With seven points this is not significant: an exact permutation test over all 5040 orderings gives a two-sided p of 0.107. The registration fixed a threshold, not a test, so the prediction held as registered, and we read it as suggestive: salience, measured this way, tracks learnability across these renderings. It was measured at one tile scale, so it does not explain why the asymmetry weakened when glyphs were spread over several patches.

---

6. Discussion

The failure is about optimisation, not representation. A donor of the right group fixes it in every seed, a wrong-group donor does no better than a cold start, both held on two sets of fresh seeds, and the probe finds nothing in a failing model that a decoder is merely unable to read. That combination is hard to explain by capacity and easy to explain by bootstrapping: there is no gradient toward the composite until

something in the model already computes part of it. A donor supplies that part, and it has to be the right group's part; which substrate it learned the group on does not matter.

Partial credit can be a trap. Handing the model a visible, correct piece of the answer did not lift it toward the whole. Runs that found the piece stopped on it, at the score it earns. The two observations fit together: a donor supplies an internal partial computation that the full task extends, while the cue supplies an external one that the task can be fit around without extending. We have not tested that reading, and it is the experiment we would most want next.

Salience tracks which pictures are learnable, and the substrate effect is smaller than it first looked. The group was learned far more often from position than from colour at the setting we started with, and the renderings whose elements are further apart in pixels are the ones learned more often. But the gap between position and colour did not survive a change of optimisation settings or of tile scale as registered, so we do not claim a general substrate law. What the picture changes may be how far apart it draws the elements to a given tokenizer, rather than whether they are positions or colours.

Looping did not help here, and the failures we study are shared by feedforward models of matched compute and matched parameters. Appendix B reports what the looped models do compute: passes track composition depth and not scene breadth, and the published halting criteria can be scored against an oracle the benchmark provides.

---

7. Limitations

- The substrate asymmetry is setting-specific. It was not shown to survive a change of learning rate and weight decay or glyphs that span several patches (Section 5.2). Everything else in the paper is at the original setting unless stated.
- The results come from one task family, one renderer and one scale, at width 384 on synthetic images. Whether any result holds at scale or on natural images is untested.
- The salience result is seven points and one measure. It is a rank correlation that cleared a registered threshold with a permutation p of 0.107, not a significant result and not a manipulation of salience, and it was measured at one tile scale.
- The trap claim is about where runs stop, not how often they escape. The composing cue's lower escape rate in the first set did not replicate.
- Held-out confirmation of the asymmetry is "not contradicted", never "confirmed". Three of the four cells have held-out seeds and none contradicts its sealed rate; the S3 position cell has none.
- Some registered tests carry no correction for multiple tests. Each was tested as registered. Under a Holm correction within each registration, the first donor-specificity set ($p = 0.048$ each) and one staircase contrast ($p = 0.018$) would not hold; the second donor-specificity set was registered with Holm and held under it.

- Some comparisons cross experiments that evaluate at different intervals, so they compare distributions and not matched seeds.
- The wrong-group donor result is a rate at ten seeds. It says a wrong-group donor does not help, not whether it harms. Three of ten fresh wrong-group recipients escaped, and why a wrong-group donor ever helps is unexplained.
- The probe evidence is from a failing run. What the appearance runs that do escape extract, and when, is unmeasured.
- Escape rates are rates at the training budget. A shorter budget would misclassify late escapers; one run escaped after step 210000.
- We do not claim looping beats matched compute. That gate failed.

Code and data

Every figure is produced by a script that reads a stored metrics file and computes nothing. Every count in the text comes from a tested generator over the stored runs, and every registered verdict from a tested module that was committed before the runs it reads had finished. The preregistration, every later registration, a ledger of each registration's commit and date, the stored metrics behind every figure and table, and the figure scripts are released with this paper, at https://github.com/K144U/What-Loops-See.

References

Bansal, A., Schwarzschild, A., Borgnia, E., Emam, Z., Huang, F., Goldblum, M. and Goldstein, T. (2022). End-to-end algorithm synthesis with recurrent networks: logical extrapolation without overthinking. arXiv:2202.05826, v3.

Chughtai, B., Chan, L. and Nanda, N. (2023). A toy model of universality: reverse engineering how networks learn group operations. arXiv:2302.03025, v2.

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J. and Kaiser, Ł. (2018). Universal transformers. arXiv:1807.03819, v3.

Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A. and Goldstein, T. (2025). Scaling up test-time compute with latent reasoning: a recurrent depth approach. arXiv:2502.05171, v2.

Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M. and Wichmann, F. A. (2020). Shortcut learning in deep neural networks. Nature Machine Intelligence, doi:10.1038/s42256-020-00257-z. arXiv:2004.07780, v5.

Greff, K., van Steenkiste, S. and Schmidhuber, J. (2020). On the binding problem in artificial neural networks. arXiv:2012.05208, v1.

Hermann, K. L. and Lampinen, A. K. (2020). What shapes feature representations? Exploring datasets, architectures, and training. arXiv:2006.12433, v2.

Hermann, K. L., Mobahi, H., Fel, T. and Mozer, M. C. (2023). On the foundations of shortcut learning. arXiv:2310.16228, v2.

Hewitt, J. and Liang, P. (2019). Designing and interpreting probes with control tasks. arXiv:1909.03368, v1.

Huang, L., Li, Y., Salehi, S., Chang, Y., Soni, A. and Kording, K. P. (2026). Formalizing the binding problem. arXiv:2606.03976, v1.

Liu, B., Ash, J. T., Goel, S., Krishnamurthy, A. and Zhang, C. (2022). Transformers learn shortcuts to automata. arXiv:2210.10749, v2.

Lu, W., Yang, Y., Lee, K., Li, Y. and Liu, E. (2025). Latent chain-of-thought? Decoding the depth-recurrent transformer. arXiv:2507.02199, v2.

Nanda, N., Chan, L., Lieberum, T., Smith, J. and Steinhardt, J. (2023). Progress measures for grokking via mechanistic interpretability. arXiv:2301.05217, v3.

Pappone, F., Crisostomi, D. and Rodolà, E. (2025). Two-scale latent dynamics for recurrent-depth transformers. arXiv:2509.23314, v2.

Popescu, A. C., Sáez de Ocáriz Borde, H. and Liò, P. (2026). Adaptive depth in looped transformers: diagnosing learned halting gates and trajectory readouts. arXiv:2607.20519, v1.

Power, A., Burda, Y., Edwards, H., Babuschkin, I. and Misra, V. (2022). Grokking: generalization beyond overfitting on small algorithmic datasets. arXiv:2201.02177, v1.

Schwarzschild, A., Borgnia, E., Gupta, A., Huang, F., Vishkin, U., Goldblum, M. and Goldstein, T. (2021). Can you learn an algorithm? Generalizing from easy to hard problems with recurrent networks. arXiv:2106.04537, v2.

Shah, H., Tamuly, K., Raghunathan, A., Jain, P. and Netrapalli, P. (2020). The pitfalls of simplicity bias in neural networks. arXiv:2006.07710, v2.

Appendix A. Provenance

Each claim, the entry in the released decision log that set up its runs, whether its test was registered before those runs existed, the module that evaluates it, and the stored output it is read from.

claim	decision log entry	test registered before its runs	module	stored output
the 2x2 of Section 5.2	preregistration, `experiment-substrate.md`	yes	`escape_rates`	the runs themselves
held-out cells of the 2x2	D-057	yes	`escape_rates --confirm`	`runs/_why_timing_20260929/confirm_register.txt`
asymmetry across optimisation settings	D-066	yes	`revision_verdicts`	`runs/_revision_verdicts_20261022/`
asymmetry at twice the tile size	D-067	yes	`revision_verdicts`	`runs/_revision_verdicts_20261021/`
looped and feedforward arms	D-032, D-037	no, a comparison of arms	`escape_rates --contrast`	`runs/_curriculum_contrasts_20261006/ffwd_arms.txt`
donor arms at ten seeds	D-040	no, confirmed by the next two rows	`escape_timing`, `escape_rates --contrast`	`runs/_why_timing_20260929/timing.txt`, `runs/_curriculum_contrasts_20261006/contrasts.txt`
donor specificity, first fresh set	D-061	yes	`curriculum_confirm`	`runs/_curriculum_confirm_20261009/d061_registered.txt`
donor specificity, second fresh set	D-069	yes	`revision_verdicts`	`runs/_revision_verdicts_20261021/`
the cue renderings, first set	D-062	yes	`why_e1e2`	`runs/_why_e1e2_20261009/`
the trap and the reversal, fresh set	D-068	yes	`revision_verdicts`	`runs/_revision_verdicts_20261021/`
salience and learnability	D-070, D-076	yes	`tile_distinguishability`	`runs/_tile_distinguishability_20261022_d076/`
early stop for added runs	D-065	yes, a protocol	the trainer	`runs/_earlystop_check_20261011/`
every other figure, and the numbers of Appendix B	D-074, D-076, D-079	not a test; Appendix B is exploratory	`scripts/build_figures.sh`	`runs/_figures_20261112_d080/`

claim	decision log entry	test registered before its runs	module	stored output
the training curves of **Figure 3** and the schematic of **Figure 2**	D-079, D-080	not a test	`f11_training`, `f12_schematic`	each run's own `metrics.parquet`; the schematic draws the renderer's tiles and reads no data; both hashed in `runs/_figures_20261112_d080/`
the worked problem of **Figure 1**, the renderings of **Figure 6**, every seed of **Figure 4** and the trap of **Figure 5**	D-080	not a test	`f13_anatomy`, `f14_renderings`, `f15_rescue`, `f16_trap`	the generator and the group's arithmetic; `runs/_tile_distinguishability_20261022_d076/d070.json`; `runs/_revision_verdicts_20261021/timing.json`; all hashed in `runs/_figures_20261112_d080/`

Appendix B. What the looped models compute

These three results were registered under the preregistration; by its own Section 0, the first and third are exploratory. A fourth, on loop-count jitter and wrong attractors, is reported in concurrent work by two of the authors, in submission, and is omitted here.

Passes track depth, not breadth. In ten seeds of family B and ten of family C, the minimum number of passes needed to reach 0.90 rises with composition depth and does not move with scene breadth. Its median is 1, 1, 2, 2, 2 and 3 at depths 1 to 6 in family B. Across breadths 12 to 16 in the same runs, pooled over depth, it stays at 2, and across breadths 4 to 8 in family C it stays at 1. Eight of the 300 family B cells never reach 0.90 within 32 passes, three at depth 5 and five at depth 6, and are left out of the medians. **Figure 10** plots both axes on a shared scale.

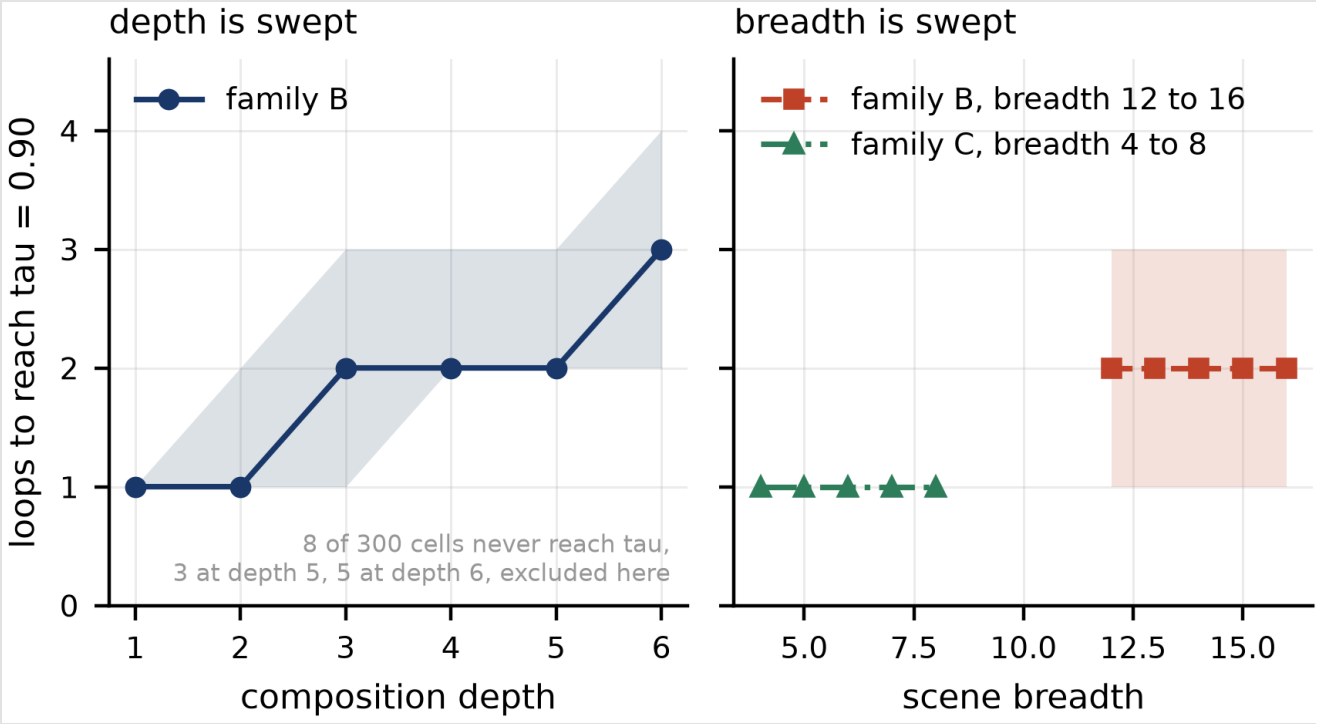


Figure 10. Minimum passes needed against composition depth and against scene breadth, on a shared scale: the median over cells, with the interquartile range shaded.

The benchmark knows how many passes each item needs, so halting criteria can be scored against that rather than against each other. Three of four criteria fire exactly at the model's own requirement, with a median signed error of 0.00. The second-order exit rule of Pappone et al. (2025) fires two loops late, at +2.00 on both families, with a sign test p of 0.0020 against a Holm threshold of 0.00357 across the fourteen tests of that family. Predictive entropy has a median error of zero and a slope of +0.24 against the requirement, firing late on deep items and early on shallow ones so that the errors cancel. **Figure 11** shows every criterion's firing point; its rows are a fixed-pass policy and an adaptive one and are not to be read across. The registered hypothesis, that the criteria are biased against the oracle and the bias grows with depth, needed one criterion with both its error and its slope rejected after Holm, and none has both. The second-order rule's error is rejected, but its slope against depth, at p = 0.0412, is not: four other tests of the family already have larger p values, so no outcome of the rest could let Holm reach it. The two criteria whose slopes are rejected, Kullback-Leibler and entropy, have a median error of exactly zero. And the hypothesis is not falsified, because an error of +2.00 is not zero. It is neither supported nor falsified, and that verdict is determinate.

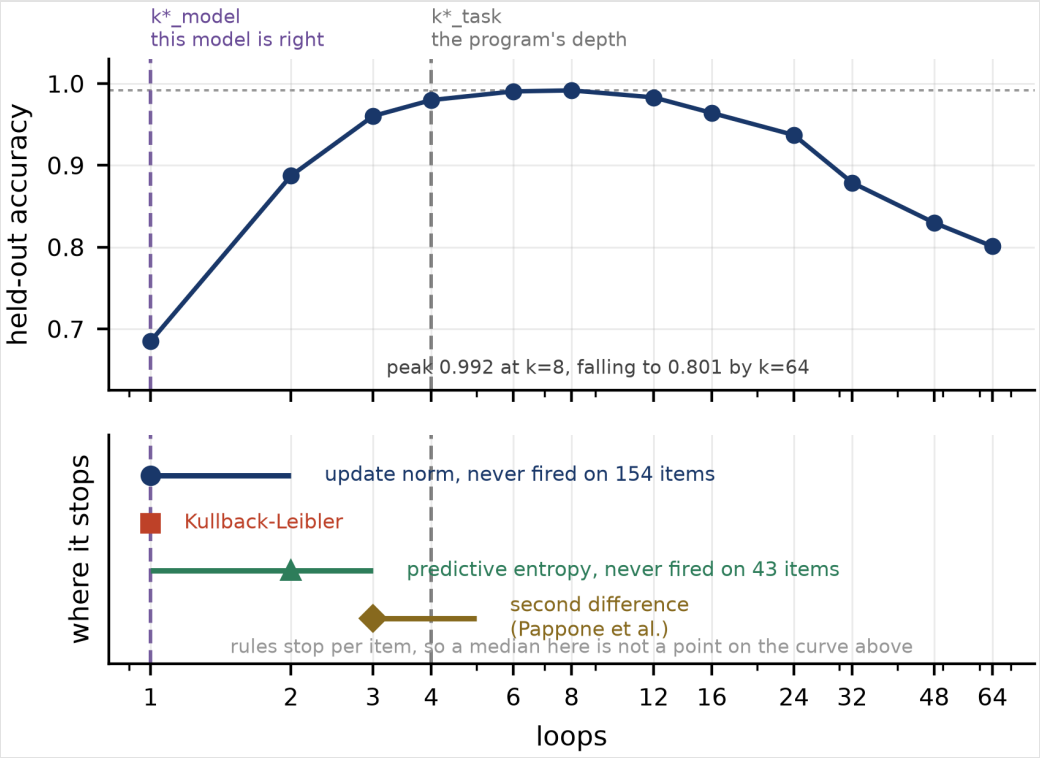


Figure 11. Accuracy against passes with both oracles and every halting criterion's firing point. The rows are a fixed-pass policy and an adaptive one and must not be compared across.

In the patching analysis, on normalised entropy, over 20 depth cells and 10 breadth cells, the loop contrast is null at -0.1021 (confidence interval -0.3062 to +0.1110) and the spatial contrast is significant at -0.1287 (-0.1870 to -0.0754) and opposite to the registered prediction. Neither survives Holm. The null loop contrast pools two depth families that sit on either side of the breadth family, family A's cells mostly below it and family B's mostly above, which **Figure 12** shows for one run of each.

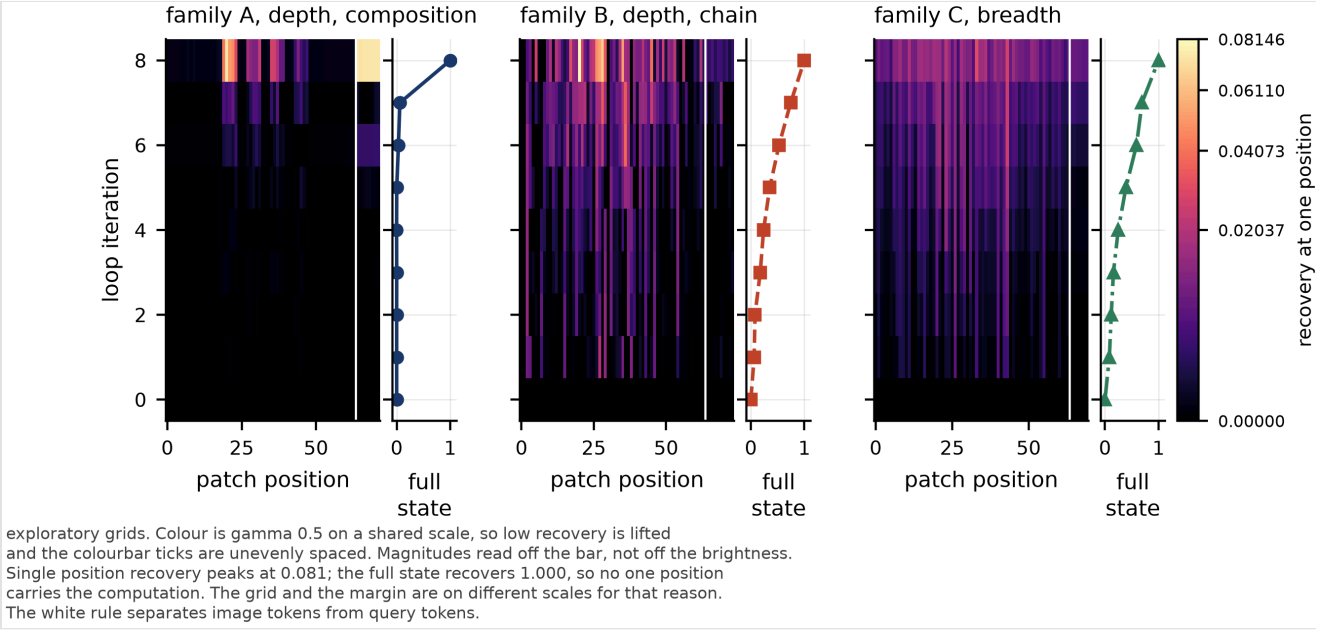


Figure 12. Patching recovery over loop iteration and patch position, one panel per family, with the recovery from patching the full state beside each.

Using the coda lens of Lu et al. (2025), on a depth 6 task the answer rises from the floor to 0.999 while the deepest intermediate never passes 0.201, which agrees with their finding of limited evidence for an interpretable latent chain of thought, here on images. **Figure 13** shows the trajectories.

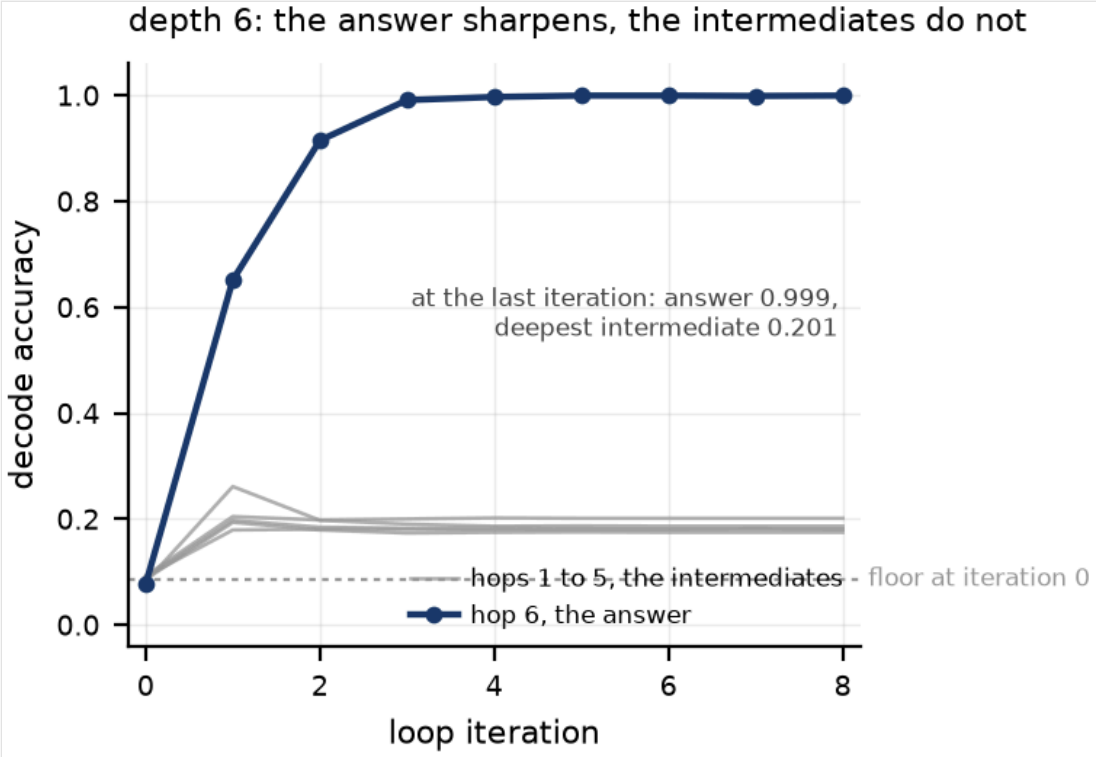


Figure 13. Coda-lens accuracy across passes on a depth 6 task, for the answer and for each intermediate composite.

Appendix C. Registered tests of why the picture matters

Two hypotheses were drawn post hoc from training curves; every test of them was registered before its runs. *H-V, a visible quotient:* composition is learned reliably when some quotient of the group is visible, because partial credit on it builds a pathway that keeps receiving gradient. *H-L, a lottery and a lock:* without such a staircase, escape happens only in an early window that then closes.

prediction	verdict
no held-out appearance seed leaves chance after step 40000	**failed**: one left at step 46350 and solved
at least 4 of 5 held-out D4 position seeds show the ln 4 plateau	held, 4 of 5
no held-out appearance seed sits on a quotient level for 10000 steps	held, none in 10
every D4 position seed with a one-character stage has it on the aspect-ratio character	**failed**: one learned the reflection first
no D4 appearance seed shows a one-character stage	**failed**: two held rotation mod 2

H-L's lock is refuted. H-V's description replicated, but its claim that the appearance rendering offers no accessible foothold did not.

The causal test used four new renderings of D4, twenty seeds each, each changing only what the tiles look like: E1a moves an F-pentomino whose bounding box is square in every orientation, so the aspect-ratio character shows nothing; E1b keeps the position tiles but assigns each to a different element; E2a and E2b are the cue renderings of Section 5.4.

	prediction	**verdict**	**numbers**
V1	E1a escapes less than D4 position	**failed**	16 of 20 against 9 of 10, p = 0.640
V2	E1a climbs less than D4 position	held	7 of 17 against 9 of 10, p = 0.018
V3	E1b escapes less than D4 position	**failed**	14 of 20 against 9 of 10, p = 0.372
V4	E1b climbs less than D4 position	**failed**	12 of 18 against 9 of 10, p = 0.364
V5	E2a, the composing cue, escapes more than E2b	**failed**	E2a 1 of 20, E2b 13 of 20, p below 0.0005
V6	E2a climbs more than E2b	held	E2a 14 of 15, E2b 0 of 14, p below 0.0005

H-V is falsified by the rule fixed in advance: if V5 failed, a visible composing quotient did not make D4 appearance learnable. Among the four accounts registered for this test, the escape pattern matched salience, which Section 5.5 then measured directly.